Don't Just Divide; Polarize and Conquer!

Shivin Srivastava
National University of Singapore
shivin@comp.nus.edu.sg

Siddharth Bhatia
National University of Singapore
siddharth@comp.nus.edu.sg

Lingxiao Huang
Tsinghua University
huanglingxiao1990@126.com

Jun Heng Lim
Nanyang Technological University
jlim294@e.ntu.edu.sg

Kenji Kawaguchi
Harvard University
kawaguch@mit.edu

Vaibhav Rajan
National University of Singapore
vaibhav@comp.nus.edu.sg

Abstract

In data containing heterogeneous subpopulations, classification performance benefits from incorporating the knowledge of cluster structure in the classifier. Previous methods for such combined clustering and classification are either 1) classifier-specific and not generic, or 2) independently perform clustering and classifier training, which may not form clusters that can potentially benefit classifier performance. The question of how to perform clustering to improve the performance of classifiers trained on the clusters has received scant attention in previous literature, despite its importance in several real-world applications. In this paper, we design a simple and efficient classification algorithm called **C**lustering **A**ware **C**lassification (CAC), to find clusters that are well suited for being used as training datasets by classifiers for each underlying subpopulation. Our experiments on synthetic and real benchmark datasets demonstrate the efficacy of CAC over previous methods for combined clustering and classification.

1 Introduction

Many real datasets have complex underlying structures such as clusters and intrinsic manifolds. For classification tasks on such data, linear classifiers fail to learn well because of the inherent non-linear data distribution. Non-linear classifiers such as (deep) neural networks perform well in such cases, but require large training data sets to achieve good generalization. When clusters are found or suspected in the data, it is well known that classification performance benefits from incorporating knowledge of such cluster structure in the model, for both linear and non-linear classifiers [1–4].

As an example, consider the problem of predicting the risk of a disease using patients' clinical data. Patient populations, even for a single disease, show significant clinical heterogeneity. As a result, subpopulations (called subtypes) having relatively homogeneous clinical characteristics can often be found in the data. Models that do not consider underlying subtypes may be consistently underestimating or overestimating the risks in specific subtypes [5], and risk models that account for subtypes have been found to be more effective [4, 6–8]. Similar examples illustrate the benefit of clustering to build classifiers in, e.g., image processing [9, 10] and natural language processing [11].

Previous works on using clustering for classification broadly fall into three categories. The first group of techniques modify specific classification techniques to account for the underlying clusters, e.g., Clustered SVMs [2]. These techniques are closely tied to the modified classifier and inherit both its

modeling strengths and weaknesses. The second group of methods attempts to cluster and train one or more classifiers simultaneously using various approaches, e.g., ensemble-based methods [3]. The third and most common approach is to first cluster the data and then independently train classifiers on each cluster. Although simple and intuitive, such a 'cluster-then-predict' approach may not form clusters in a manner that benefits the performance of classifiers trained on those clusters. This is also a limitation of techniques in the second group where clustering and classifier training occur independently. To the best of our knowledge, the question of performing clustering to improve the performance of classifiers trained on the clusters has not been addressed by previous literature.

Thus, in this paper, we first investigate the fundamental question of when and how clustering can help in obtaining accurate classifiers. Our analysis yields novel insights into the benefits of using simple classifiers trained on clusters compared to simple or complex (in terms of Rademacher complexity [12]) classifiers trained without clustering. It provides clues on when such a clustering-based approach may not improve the subsequent classification. Finally, it also motivates the design of our clustering-based framework for classification, called Clustering Aware Classification (CAC).

The CAC framework can be viewed as a classification algorithm that, by design, finds clusters that are intended to be used as training datasets by the (base) classifiers of choice e.g., Logistic Regression (LR), SVM or Neural Networks. The key idea of our approach is to induce class separability (polarisation) within each cluster. This is effected by designing a cost function that can be used within the optimization framework of k-means. However, finding clusters with this cost function using the popular Lloyd's algorithm may not guarantee convergence. Therefore, we design an approach based on Hartigan's method [13] and prove that it converges. Our design is generic and not classifier-specific, and leads to an efficient iterative method to find the clusters. We also theoretically investigate the benefits offered by class separation to classifiers employing the standard log-loss function. We find that the upper and lower bounds for the standard log-loss of a dataset are linearly dependent on class separation within the dataset. We analyze the sensitivity of CAC to various data characteristics such as class separation, intercluster distance, and the number of clusters using synthetic data. Finally, we empirically study the performance of CAC on benchmark datasets, including a large clinical dataset. We find that CAC leads to improved classification performance (by 4.63%) when compared to previous methods for combined clustering and classification.

In summary, our contributions are:

1. Our theoretical analysis provides insights into the fundamental question of how clustering can aid in potentially improving the performance of classifiers trained on the clusters.
2. We design a novel algorithmic framework, Clustering Aware Classification (CAC) to find clusters in data with the aim of improving the performance of classifiers trained on the clusters. CAC is simple, efficient, and provably convergent. It can be used with any classifier.
3. We analyze the sensitivity of CAC to various algorithm parameters and cluster characteristics in the data using synthetic data. Our findings lend support to our theoretical analysis.
4. The experimental results on benchmark datasets including a large clinical dataset, show that CAC yields higher improvement in classification performance compared to previous approaches for combined clustering and classification.

Reproducibility: Our code and datasets are publicly available at this link.

2 Related Work

The idea of training multiple local classifiers on portions of the dataset has been extensively studied for specific classifiers [14–21]. For instance, [22] presents a novel locality-sensitive support vector machine (LSSVM) for the image retrieval problem. Locally Linear SVM [23] has a smooth decision boundary and bounded curvature. Its authors show how functions defining the classifier can be approximated using any local coding scheme. [24] proposes Mixed SVMs which partition the feature space into subregions of linearly separable data points and learn a Linear SVM for each of these regions. [2] proposes Clustered Support Vector Machines (CSVMs) which tackles the problem in a divide-and-conquer manner, thus avoiding the need for computationally expensive kernel SVMs. CSVM groups the data into several clusters, after which it trains a linear SVM in each cluster to separate the data locally. Additionally, it imposes a global regularization, that aligns the weight vectors of each locally linear SVM with a global weight vector.

There are other related works, which aim to find suitable representations of raw data such that the performance in downstream prediction tasks is better when they use the learned embeddings. Large Margin Nearest Neighbour (LMNN) [25] introduces the much-celebrated triplet loss. In this work, the authors find a data-specific Mahalanobis distance embedding for the data, subject to the constraint that k similarly labeled points are mapped in the neighborhood of every point. The transformed data is more amenable for a k Nearest Neighbour classifier to work with. [26] extends LMNN to make it parameter-free. Recently, there has been a surge of interest in contrastive learning [27, 28] as well where the core idea is to group similar/dissimilar representations of data into similar/dissimilar groups. The latent representations thus generated are beneficial to supervised classifiers in classification tasks.

Some Bayesian approaches have been developed for combined classification and clustering, e.g., [1, 29] but they are computationally expensive. To the best of our knowledge, the closest and most recent work in spirit and purpose to our work are the EC3 [3] and DMNN [4]. EC3 is based on a systematic combination of multiple classification and clustering methods combined via a convex optimization function. It uses 6 base classifiers: Decision Tree, Naive Bayes, K- nearest neighbor, logistic regression, SVM, and stochastic gradient descent classifier. It also considers 3 clustering methods DBSCAN, Hierarchical, and k-Means.

The Deep Mixture Neural Network (DMNN), consisting of an embedding network with gating and a user-defined number of local predictive networks proposes simultaneous patient subtyping and risk prediction. The gating network computes high-level feature representations of raw input features, which are then clustered to find patient cohorts. Local predictor networks are then trained to predict the outcomes for their respective cohorts. The subgroup-specific sets of features learned by the local features provide valuable information from which we can reason about the cluster phenotype. Thus, DMNN represents the 'cluster-then-predict' type of approach as it uses local predictor networks to classify subpopulations in the data.

3 Problem Statement

Let $X = (x_1, x_2, \cdots, x_N)$ be a training dataset with n records $x_i \in \mathbb{R}^d$ together with binary labels $y_i \in \{0, 1\}$. Given an integer $k \geq 1$, our goal is to:

1. Divide X into k non-empty, distinct clusters $\mathcal{C} = \{C_1, C_2, \cdots, C_k\}$, and
2. Obtain a collection of k classifiers $\mathcal{F} = \{f_1, f_2, \cdots, f_k\}$ where each $f_j : C_j \to \{0, 1\}^{|C_j|}$ is trained on all $x_i \in C_j$ with the objective $\text{argmin}_{\mathcal{F},\mathcal{C}} \sum_{j=1}^{k} \sum_{i \in C_j} \ell(f_j(x_i), y_i)$.

There is no restriction on the choice of the classifier, which is determined apriori by the user.

4 Can Clustering Aid Classification?

We provide a principled motivation behind our clustering approach by analysing when clustering can potentially help in obtaining accurate classifiers. Our analysis yields crucial insights into developing the CAC framework.

Fix $j \in \{1, \ldots, k\}$ and let $S_j = \{(x_i, y_i)\}_{i=1}^{m_j}$ be the training dataset from the j-th cluster (having m_j points) with $x \in \mathcal{X}_j$ and $y \in \mathcal{Y}_j$ s.t. $\mathcal{X}_j = \{x : x \in C_j\}$ and $\mathcal{Y}_j = \{y : y \in C_j\}$. Then, the following theorem — a direct application of a previous result [12, 30] to our problem — suggests a potential benefit of clustering in terms of the expected error $\mathbb{E}_{x,y}[\ell(f(x), y)]$ for unseen data:

Theorem 1. *Let $\mathcal{F}_j$ be a set of maps $x \mapsto f(x)$, and the function $q \mapsto \ell(q, y)$ be a λ-uniformly bounded function for any $q \in \{f(x) : f \in \mathcal{F}_j, x \in \mathcal{X}_j\}$ and $y \in \mathcal{Y}_j$. Then, for any $\delta > 0$, with probability at least $1 - \delta$ over an i.i.d. draw of m i.i.d. samples $((x_i, y_i))_{i=1}^{m}$, the following holds: for all maps $f \in \mathcal{F}_j$,*

$$\mathbb{E}_{x,y}[\ell(f(x), y)] \leq \frac{1}{m_j} \sum_{i=1}^{m_j} \ell(f(x_i), y_i) + 2\mathcal{R}_{m_j}(\ell \circ \mathcal{F}_j) + \lambda \sqrt{\frac{\ln(1/\delta)}{2m_j}}, \tag{1}$$

where $\mathcal{R}_{m_j}(\ell \circ \mathcal{F}_j) := \mathbb{E}_{S,\sigma}[\sup_{f \in \mathcal{F}_j} \frac{1}{m_j} \sum_{i=1}^{m_j} \sigma_i \ell(f(x_i), y_i)]$ where $\sigma_1, \ldots, \sigma_{m_j}$ are independent uniform random variables taking values in $\{-1, 1\}$.

Proof. We present the proof in Appendix A.1. □

Theorem 1 shows that the expected error $\mathbb{E}_{x,y}[\ell(f(x), y)]$ for unseen data is bounded by three terms: the training error $\frac{1}{m_j}\sum_{i=1}^{m_j}\ell(f(x_i), y_i)$, the Rademacher complexity $\mathcal{R}_{m_j}(\ell \circ \mathcal{F}_j)$ of the set of classifiers $\mathcal{F}_j$ on the j-th cluster, and the last term $O(\frac{\ln(1/\delta)}{2m_j})$ in equation (1). To apply Theorem 1 to models without clustering, we can set the j-th cluster to contain all the data points with $m_j = N$.

The upper bound can be used to analyze the benefit of using a simple model (e.g., a linear model) *with* clustering compared to two cases of (1) a simple model and (2) a complex model (e.g., a deep neural network), both *without* clustering. For a simple model *without* clustering, the training error term $\frac{1}{m_j}\sum_{i=1}^{m_j}\ell(f(x_i), y_i)$ in equation (1) can be large, because a simple model may not be able to sufficiently separate training data points (underfitting) to minimize the training error. In the case of a complex model *without* clustering, the training error can be small but the Rademacher complexity $\mathcal{R}_{m_j}(\ell \circ \mathcal{F}_j)$ tends to be large. A simple model *with* clustering can potentially trade-off these by minimizing the Rademacher complexity $\mathcal{R}_{m_j}(\ell \circ \mathcal{F}_j)$ while making the training data in the cluster to be linearly separable to minimize the training error $\frac{1}{m_j}\sum_{i=1}^{m_j}\ell(f(x_i), y_i)$. This observation leads us to explore ways to induce class separability in data clusters.

Theorem 1 also shows that such a clustering-based approach may also not help in some cases in improving classification. Using the clustering approach decreases the value of m and thus increases the last term $O\left(\frac{\ln(1/\delta)}{2m}\right)$ in Equation (1), when compared with models without clustering. Therefore, using a simple model with clustering can be potentially advantageous when the first and second terms in Equation (1) are dominant, but may not be beneficial if the last term is dominant.

Here, the Rademacher complexity term $\mathcal{R}_{m_j}(\ell \circ \mathcal{F}_j)$ is also dependent on the sample size of the cluster m_j and it can be typically bounded by $O(\sqrt{\frac{M_j}{m_j}})$ for both linear models and deep neural networks where M_j is a factor that depends on F_j [12, 30–32]. For example, [12, 30, 33] quantify the Rademacher complexity of linear classifier by $O(\sqrt{\frac{M}{m}})$ with $M = \|w\|$ where w is the weight of the linear classifier $w^\top x$. [31] quantify the Rademacher complexity of deep neural netwroks by $O(\sqrt{\frac{M}{m}})$ where M is the products of the spectral norms of weights at each layers. [32] quantifies the Rademacher complexity of deep neural networks by $O(\sqrt{\frac{M}{m}})$ where M is a path norm of weights.

Since large m reduces both the second and third terms, unsurprisingly, larger sample sizes may lead to improved performance. The theorem shows that simple models within each cluster may lead to good generalization due to smaller values in the first two terms. It thus motivates the formation of clusters, each of which can be linearly separable internally with respect to the classes. We further observe the implications of Theorem 1 in section 6 in the light of empirical results.

5 CAC: Our Algorithm

In this section, we develop our framework Clustering Aware Classification (CAC). It can be viewed as a classification algorithm that by design finds clusters that are intended to be used as training datasets by classifiers. The key idea of our approach is to induce class separability within each cluster. If the points of different classes are well separated within a cluster, then it naturally aids any classifier in determining the class boundaries.

5.1 Class Separability

We use a simple heuristic to measure class separability within each cluster. For each cluster C_j ($j \in [k]$), we define: *cluster centroid*: $\mu(C_j) := |C_j|^{-1}\sum_{x \in C_j} x_i$, *positive centroid*: $\mu^+(C_j) := \frac{1}{\sum_{x_i \in C_j} y_i}\sum_{x_i \in C_j} y_i x_i$, and *negative centroid*: $\mu^-(C_j) := \frac{1}{\sum_{x_i \in C_j}(1-y_i)}\sum_{x_i \in C_j}(1-y_i)x_i$.

Class separability is defined by considering the distance between positive and negative centroids within each cluster. Figure 1 shows two clusters C_1 and C_2. C_1 has greater class separation while in

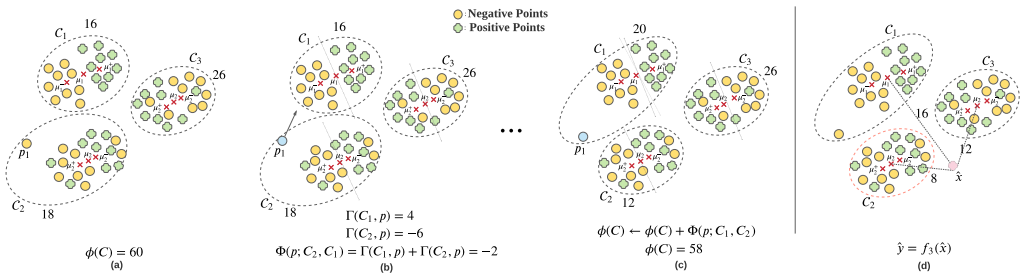


Figure 1: CAC problem setting (with dummy values for illustrative purposes): (a) The total CAC cost of this clustering is 60; (b) Point p_1 is selected to be reassigned to cluster C_1 based on the cluster update equations; (c) p_1 is assigned to C_1 and the cost functions of C_1 and C_2 are updated; (d) At testing time, $\hat{x}$ is assigned to cluster C_2 as it lies nearest to $\hat{x}$.

C_2, the data points are more intermingled and thus their centroids are closer. Thus, $\|\mu_1^+ - \mu_1^-\| > \|\mu_2^+ - \mu_2^-\|$.

We note that the current definition of class separability can only incorporate two classes. Extending CAC to multi-class setting can be considered in future work.

5.2 Clusters with Class Separability

To induce our notion of class separability within clustering we consider the k-means clustering formulation that is widely used for its simplicity, effectiveness, and speed. Clusters using k-means are obtained by minimizing the size-weighted sample variance of each cluster: $\mathcal{D}(X) = \sum_{p=1}^{k} \inf_{\mu(C_p) \in X} \sum_{i \in C_p} \|x_i - \mu(C_p)\|^2$.

Thus, the cost of each cluster, given by $\sum_{i \in C_j} \|x_i - \mu(C_j)\|^2$, is minimized. To induce class separability, a natural formulation is to add a cost with respect to class separability for each cluster. With our measure for class separability, and $\alpha > 0$ as a class separation hyperparameter, this becomes:

$$\phi(C_j) = \sum_{i \in C_j} \|x_i - \mu(C_j)\|^2 - \alpha \cdot \|\mu^+(C_j) - \mu^-(C_j)\|^2.$$

Defining the cost function in this manner encourages the decrease of cluster variance term and increase of the class separability term, weighted by α. Consequently, the overall cost function is defined to be $\phi(\{C_j\}_1^k) := \sum_{j=1}^{k} \phi(C_j)$ as the *unsupervised* loss of CAC that we aim to minimize.

5.3 Finding the clusters

The most common heuristic to find k-means clusters is Lloyd's Algorithm [34]. It begins with an initial clustering and iteratively computes cluster centroids and assigns points to clusters corresponding to their closest centroids. However, using this heuristic with our proposed cost may not guarantee convergence. (See our discussion after Theorem 2). Instead, we adopt an approach based on Hartigan's method [13]. For k-means clustering, Hartigan's method proceeds point by point in a greedy manner. Each point is reassigned to a cluster such that the overall cost is reduced.

In our setting, denote the cost of merging a point x to some cluster C_q as $\Gamma^+(C_q, x) = \phi(C_q \cup \{x\}) - \phi(C_q)$. Symmetrically, denote the cost of removing a point x from some cluster C_p as $\Gamma^-(C_p, x) = \phi(C_p \setminus \{x\}) - \phi(C_p)$. Consequently, define the improvement in cost by moving a point $x \in C_p$ from cluster C_p to cluster C_q to be $\Phi(x; C_p, C_q) := \Gamma^+(C_q, x) + \Gamma^-(C_p, x)$. The exact expressions for $\Gamma^+(C_q, x)$ and $\Gamma^-(C_p, x)$ w.r.t $x, \mu, \mu^{\pm}$ are derived in Appendix A.2.

Our approach is summarized in Algorithm 1. It begins with an initial clustering (k-means found clusters) and then proceeds iteratively in rounds. In each round, all points are checked to determine if a point has to be re-assigned to another cluster. For a point in the current iteration, x_i, if moving $x_i \in C_p$ would result in C_p/x_i having points of only one class, then it is not moved. If not, the

cluster C_q that results in the maximum decrease in cost $\Phi(x_i; C_p, C_q)$ is found and x_i is moved from its current cluster C_p to C_q.

In each round, all three centroids for each cluster are updated and once the loss function has become stagnant, the algorithm is considered to have converged. Base classifiers are then trained on the clusters found and are used for predicting labels for the unseen test data.

Algorithm 1: CAC

Input: Training Data: $X \in \mathbb{R}^{n \times d}$, binary labels $y^{n \times 1} \in \{0,1\}^n$, $\{C_j\}_1^k$, $\{f_j\}_1^k$ and α
Output: k Trained Classifiers, k cluster centers

▷ **Initialization**
Compute $\mu(C_j), \mu^+(C_j), \mu^-(C_j)\ \forall\ C_j\ s.t.\ j \in \{1, \cdots, k\}$.
▷ **Algorithm**
while *not converged* **do**
  for $i \in \{1 \dots n\}$ **do**
    if *Removing* $x_i \in C_p$ *from its cluster does not lead to a* 1*-class cluster* **then**
      ▷ **Assign new cluster to** x_i
      Let $q := \text{argmin}_j\ \Phi(x_i; C_p, C_j)$ (breaking ties arbitrarily).
      if $\Phi(x_i, C_p, C_q) < 0$ **then**
        Assign x_i to C_q from C_p and update $\mu(C_p)$, $\mu(C_q)$, $\mu^{\pm}(C_p)$ and $\mu^{\pm}(C_q)$.
      Otherwise, let x_i remain in C_p.
▷ **Train Classifiers**
For every cluster C_j, train a classifier f_j on $(\mathcal{X}_j, \mathcal{Y}_j)$.
Output Classifiers $\mathcal{F} = \{f_j\}_{j=1}^{j=k}$ and cluster centroids $\mu = \{\mu_j\}_{j=1}^{j=k}$

We now show that CAC is guaranteed to converge and prove that its per-round time complexity is linear in the number of data points, dimensions, and number of clusters.

Theorem 2. *Algorithm 1 converges to a local minimum.*

Proof. To prove convergence, it suffices to verify that the cost function, that re-assigns point x from cluster C_p to cluster C_q, monotonically decreases at each iteration. This is equivalent to showing $\phi(C_p \setminus \{x_i\}) + \phi(C_q \cup \{x_i\}) < \phi(C_p) + \phi(C_q) \implies (\phi(C_p \setminus \{x_i\}) - \phi(C_p)) + (\phi(C_q \cup \{x_i\}) - \phi(C_q)) < 0 \implies \Gamma^+(C_q, x) + \Gamma^-(C_p, x) < 0 \implies \Phi(x_i, C_p, C_q) < 0$.

$\Phi(x_i, C_p, C_q) < 0$ is exactly the update condition for the algorithm that ensures that the total cost function is always decreasing with every point update and in every iteration. □

Theorem 3. *The time complexity for each round of CAC is* $O(ndk)$.

Proof. We present the proof in Appendix A.2. □

Remark 1 (Hartigan vs. Lloyd). For k-means clustering, both Lloyd's method and Hartigan's method converge since the cost function always decreases monotonically with each update. Note that Lloyd's method does not consider the change in centroids while updating the assignment of a point, and instead, directly assign points to their current closest centroids. In our setting, Lloyd's method can similarly fix all centroids, compute a new assignment, and update the centroids accordingly. For instance, replacing the assignment rule of x_i in Algorithm 1 by $q := \text{argmin}_j\ \|x_i - \mu(C_j)\|^2 - \alpha \cdot \|\mu^+(C_j) - \mu^-(C_j)\|^2$. However, our cost function contains an additional squared distance term between positive and negative centroids, i.e., $-\alpha \cdot \|\mu^+(C_j) - \mu^-(C_j)\|^2$. There is no guarantee that this term monotonically decreases by Lloyd's method during the update of $\mu^+(C_j)$ and $\mu^-(C_j)$ after an assignment. Hence, it is non-trivial to ensure and prove the convergence of Lloyd's method with our cost function. However, with the use of Hartigan's method, convergence is ensured by design. As for the rate of convergence, empirical experiments demonstrate that the rate of convergence is positively related to α (See Figure 4b in Appendix B.3).

Remark 2 (Class separation). We now ask the question, whether CAC ensures increase in class separation within clusters at the end of the clustering process; or more strongly, with every iteration?

Empirical experiments on synthetic and real datasets (see Figure 4a in B.3) indicate that this is indeed true. Let us rewrite the CAC cost function as follows. $\phi(X) = \mathcal{D}(X) - \alpha \cdot \mathcal{S}(X)$ where

$$\mathcal{D}(X) = \sum_{j=1}^{k} \sum_{i \in C_j} \|x_i - \mu(C_j)\|^2; \;\; \mathcal{S}(X) = \sum_{j=1}^{k} \sum_{i \in C_j} \|\mu^+(C_j) - \mu^-(C_j)\|^2$$

Since we initialize CAC with k-means clustering, $\mathcal{D}_0(X)$ is at a local minima, where the subscript denotes the iteration. An immediate consequence of this is that in the first iteration, $\mathcal{D}_1(X) > \mathcal{D}_0(X)$ and $\mathcal{S}_1(X) > \mathcal{S}_0(X)$ as $\phi_1(X) < \phi_0(X)$. Intuitively it seems logical that running Hartigan's method with the CAC loss should lead to a higher k-means loss i.e. $\mathcal{D}_\infty(X) > \mathcal{D}_0(X)$ as otherwise there arises an intriguing scenario. Suppose that the opposite is true i.e. $\mathcal{D}_\infty(X) < \mathcal{D}_0(X)$, then instead of running the clustering algorithm with the standard SSE loss function ($\mathcal{D}$), it would be better if we use the CAC loss function i.e. $\mathcal{D} - \alpha \cdot \mathcal{S}$ in the first place. We thus conjecture that running Hartigan's algorithm with the CAC loss function ultimately leads to increased class separation in clusters and a higher k-means cost as well i.e. $\mathcal{D}_\infty(X) > \mathcal{D}_0(X)$ and $\mathcal{S}_\infty(X) > \mathcal{S}_0(X)$.

5.4 CAC prediction

CAC outputs a set of cluster representatives denoted by their centroids and the trained classifiers associated with those clusters. To make predictions for a test point $\hat{x}$, the first step is to find the cluster to which the test point is most likely to belong. This can be done by assigning the test point to a cluster represented by the 'closest' centroid C_j in the data space i.e. $j = \text{argmin}_l \|\hat{x} - \mu_l\|^2$. The corresponding classifier f_j can then be used to predict $\hat{y} = f_j(\hat{x})$.

5.5 Bounding Log-Loss of Classifiers in terms of Class Separation

We now prove how class separability within clusters has a direct effect on the training error of classifiers employing log-loss. The following theorem also validates our choice of using $\|\mu^+ - \mu^-\|$ as a heuristic to measure class separability.

Lemma 1. *For d dimentional vectors $\mathbf{u}, \mathbf{v}$ and $\boldsymbol{\beta}$, $\frac{\partial \Delta}{\partial \|\mathbf{u}\|} < 0$ for $\Delta = A - (B\boldsymbol{\beta}\mathbf{u} + C\boldsymbol{\beta}\mathbf{v})$ if $\beta_i u_i > 0, \forall i \in \{1, 2, \cdots, d\}$ and $A, B > 0$.*

Proof. We present the proof in Appendix A.3. □

Theorem 4. *Let $X = (x_1, x_2, \cdots, x_N)$ be a training dataset with n records $x_i \in \mathbb{R}^d$ together with binary labels $y_i \in \{0, 1\}$. Define the log-loss over the entire dataset as : $\ell(X) = -\Sigma_i y_i \ln(p(x_i)) + (1 - y_i)\ln(1 - p(x_i))$, where $p(x_i) = [1 + \exp(-\beta x_i)]^{-1}$ and $\beta = \text{argmin}_\beta \, \ell(X)$. Then,*

$$C_1 - (C_3\beta\mu^+ - C_4\beta\mu^-) \leq \ell(X) \leq C_2 - (C_3\beta\mu^+ - C_4\beta\mu^-)$$

where $C_1 = N\ln(2)$, $C_2 = N\ln(1 + \exp(c)) - \frac{Nc}{2}$, $C_3 = \frac{N^+}{2}$, $C_4 = \frac{N^-}{2}$, $N^+ = \sum_{x_i \in X} y_i$, $N^- = \sum_{x_i \in X}(1 - y_i)$ and $c = \text{argmax}_i \|\beta x_i\|$.

Proof. We present the proof in Appendix A.4. □

Note that the difference between the upper (UB) and lower bound (LB) for $\ell(X)$ is independent of the class separation parameter and depends only on c, the maximum distance that a point in the cluster is from the regression line. $UB - LB \leq \frac{c^2}{4}$ (considering the Taylor series expansion of C_2). We can thus approximate $\ell(X)$ by $\Delta = A - (B \cdot \beta\mu^+ - C \cdot \beta\mu^-)$ for some $A, B, C > 0$. To show that Δ decreases with increasing class separation, we need to prove that $\frac{\partial \Delta}{\partial \|\mu^+ - \mu^-\|} < 0$. Changing the variable space to $\mathbf{u} = \mu^+ - \mu^-$ and $\mathbf{v} = \mu^+ + \mu^-$ ($\mathbf{u}$ and $\mathbf{v}$ being linearly independent of each other), we get $\Delta = A - [(B' + C')\beta \cdot \mathbf{u} + (B' - C')\beta \cdot \mathbf{v}]$ where $B' = \frac{B}{2}$ and $C' = \frac{C}{2}$. From Lemma 1, we know that $\frac{\partial \Delta}{\partial \|\mu^+ - \mu^-\|} < 0$ if $\beta_i \cdot (\mu_i^+ - \mu_i^-) > 0, \forall i$. Thus we show that if the class centroids lie entirely (in every dimention) on one side of the regression plane, increasing class separation decreases the upper and lower bounds of the log-loss error.

6 Experiments

We perform two sets of experiments. The first set of experiments studies, on synthetic data, study the sensitivity of CAC on various characteristics of the data. In the second set of experiments, we compare the performance of CAC with that of other methods on real benchmark datasets. All experiments are conducted on a 2.6GHz Intel $i7$ CPU with 16 GB RAM running MacOS 11.1.

6.1 Experiments on Synthetic Data

We analyze how the performance of CAC depends on the following data/algorithm parameters:

1. Inner Class Separation (ICS): Distance between class centroids in an individual cluster.
2. Outer Cluster Separation (OCS): Distance between cluster centroids.
3. Number of natural clusters in the dataset (K).
4. Number of classifiers used in CAC (k), i.e., the number of clusters used as input.

To study the performance of CAC as a function of these 4 parameters, we modify the `make classification` function of *sklearn* library [35], to generate custom datasets with varying ICS, OCS, and K. We find that the performance of CAC improves with increasing ICS. This is in line with our hypothesis that if the classes are well separated within the clusters, then it will improve the performance of the classifiers. The performance of CAC does not depend much on the OCS. As the number of natural clusters (K) in the dataset increases, the performance of CAC decreases. On the other hand, the performance is better when $k > K$. Appendix B.1 presents the details of synthetic experiments and the results.

6.2 Experiments on Real Data

Datasets. We use 5 benchmark binary-labelled datasets that were used to evaluate the EC3 algorithm in [3]. Table 1 presents a summary. We follow the data preprocessing steps given in [3].

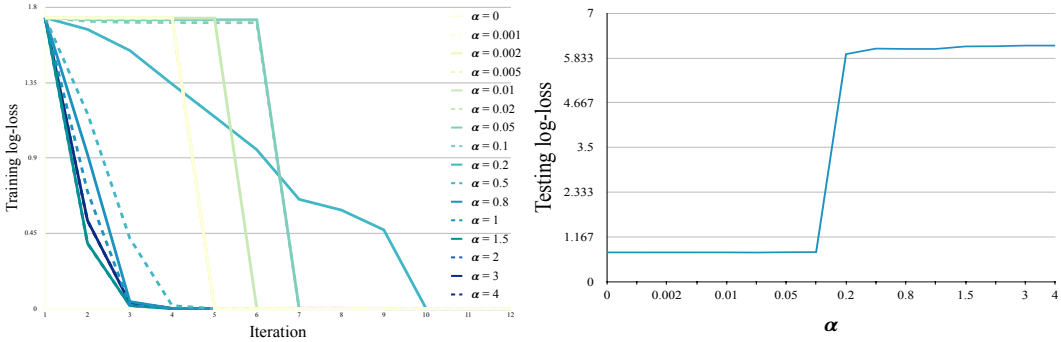
Table 1: Summary of binary classification datasets. MAJ: proportion of majority class instances.

Dataset	#Instances	#Features	MAJ
Titanic [36]	2200	6	0.68
Magic [37]	19020	10	0.64
Creditcard [38]	30000	24	0.78
Adult [37]	45000	14	0.75
Diabetes [37]	100000	9	0.54

In addition, we use a large, clinical dataset, that we call CIC, from the 2012 Physionet challenge [39] of predicting in-hospital mortality of intensive care units (ICU) patients at the end of their hospital stay. The data has time-series records, comprising various physiological parameters, of 12,000 patient ICU stays. We follow the data processing scheme of [40] (the top ranked team in the competition) to obtain static 117-dimensional features for each patient.

Experimental Setup. For each dataset we randomly choose 25% of the data as held-out data on which classifier performance is evaluated. The remaining data is used for training the classifiers and performing GridCV parameter search. Standard binary classification metric, the F1 score, is used to evaluate classifier performance. Results on 5-fold CV are reported in Appendix B.4, Tables 4 and 5.

Baselines. Two state-of-the-art baseline methods for combined clustering and classification are EC3 [3] and DMNN [4]. EC3 has been shown to outperform 12 other classifiers on these datasets, including popular classifiers like Random Forest and AdaBoost. It also compares with BGCM [41] and UPE [42], two consensus maximization approaches that ensemble both classification and clustering methods. DMNN originally employs the k-means algorithm for clustering and a neural network based classifier. In addition, we use 7 different base classifiers, implemented by *sklearn* library (BSD licenced): LR, Linear SVM, Linear Discriminant Analysis (LDA), single Perceptron, Random Forest (#trees = 10), k Nearest Neighbors ($k = 5$), SGD classifier and Ridge classifier (Ridge regressor predicting over the range $[-1, 1]$) as a baseline directly and with k-means (KM+X)



(a) Training Log-Loss vs iteration.

(b) Testing Log-Loss vs α

Figure 2: CAC Training and Testing curves on Credit dataset with varying α ($k = 3$) and LR base classifier. The minimum testing log-loss is achieved at $\alpha = 0.02$.

in a 'cluster-then-predict' approach. CAC is also combined with DMNN where k-means is replaced with CAC, and DMNN's underlying neural architecture is used. FNN is base DMNN but w/o any clustering. We implement DMNN ourselves and use the EC3 implementation by [43].

Hyperparameters. For all KM+X and CAC+X experiments in Table 2, we choose $k = 2$ ($k = 3$ for Creditcard dataset). For CAC, we find the optimal value of α by performing a GridCV search over the range $[0.01, 3]$. We evaluate the algorithm on a separate held-out test dataset. For DMNN, the autoencoder has 3 layer of sizes $64 - 32 - 64$ and a two-layered local predictor network of size $30 - 1$. The autoencoder has L1-L2 regularization (L1: 1e-5, L2: 1e-4) and no activation. The local prediction networks use softmax and relu activation functions and are not regularized. Details are in Appendix B.2.

Setting α parameter. The performance of CAC heavily depends on the value of α. We illustrate this in Figure 2a for one dataset. If α is too small, then the training will be very slow, and the clusters found will not have enough class separability. If α is too large, CAC will show overfitting behavior (Figure 2b). Although the clusters formed will have large class separability and the training error of base classifiers will approach 0, because the clusters are too spread out, at test time, the performance will be poor.

Results. Table 2 presents the F1 scores obtained by all algorithms.

- We observe that the average % increase in F1 score of CAC (k-means + classifier) over the base classifier ranges from 1.51% (0.93%) for Magic dataset to 91.5% for the Diabetes dataset (15.07% for creditcard dataset) (values not in Table 2). This is in line with theorem 1 that predicts better performance of multiple simple classifiers as compared to a single simple classifier.
- EC3 (consisting of 6 classifiers and 2 clustering algorithms) outperforms the cluster-then-predict schemes on the Creditcard and CIC datasets. On all other datasets, there exists a CAC variant that outperforms it. Thus we find evidence for using clustering analysis in a cluster-then-predict instead of combining the predictions of clustering and classification algorithms as done by EC3. The best performing CAC classifiers (across datasets) perform on an average 4.63% better than EC3.
- Except for one dataset (Magic), CAC out performs the simple k-means approach on all the datasets both in terms of best F1 and average $\uparrow$ F1 (by a wide margin of 0.6 to 78%). CAC also performs exceptionally well on the Diabetes dataset. We find that the Inter Class Separation (ICS) plays an important role in determining the performance of CAC. See Figure 5 in Appendix C.
- CAC performs better than the simple k-means + classification approach on 5 out of 6 datasets. This serves as an ablative evidence of the contribution of linear separability criteria since removing the latter reduces CAC to applying k-means and then predicting.
- EC3 consistently took more time than CAC to execute on all datasets. We note that its running time is proportional to k^2 while CAC's running time is proportional to k (the number of clusters).

Table 2: F1 of each method on different datasets evaluated on a separate held out testing dataset. $*/*/*$ indicates the number of times the base classifier, KM+Base and CAC+Base attains the best performance. Highlighted values signify the best result in that category while the overall best classifier for a dataset is denoted by *. The Avg. $\uparrow$ F1 denotes the average % increase in F1 score of CAC over Base and k-means variants (overall best CAC variant for EC3) across different classifiers (row-wise) and datasets (column-wise).

Classifier	Titanic	Magic	Credit	Adult	Diabetes	CIC	Total	Avg. $\uparrow$ F1(%)
EC3	0.848	0.858	**0.512***	0.660	NA	**0.498***	2	4.63
LR	**0.839**	0.852	0.336	0.556	0.12	0.45		20.81
KM + LR	0.674	**0.861**	0.505	0.619	**0.503***	**0.468**	1/3/2	4.9
CAC + LR	0.838	0.859	**0.506**	**0.642**	0.497	0.467		–
SVM	0.84	0.852	0.249	0.532	0.112	0.412		25.06
KM + SVM	0.862	**0.879**	0.398	0.601	0.125	0.44	0/1/5	13.4
CAC + SVM	**0.862***	0.878	**0.447**	**0.62**	**0.497**	**0.444**		–
LDA	0.828	0.846	0.358	0.531	0.12	0.446		19.69
KM + LDA	0.857	**0.87**	**0.423**	0.592	0.105	**0.487**	0/3/3	12.33
CAC + LDA	**0.857**	0.87	0.421	**0.62**	**0.497**	0.48		–
Perceptron	0.729	0.83	0.367	0.482	0.469	0.378		13.7
KM + Perceptron	**0.837**	0.8	0.329	0.579	0.407	0.421	0/1/5	9.76
CAC + Perceptron	0.831	**0.85**	**0.457**	**0.62**	**0.497**	**0.446**		–
RF	0.836	**0.896**	0.409	0.659	0.413	0.348		6.09
KM + RF	0.835	0.886	0.396	0.655	0.409	0.343	1/0/5	7.21
CAC + RF	**0.839**	0.89	**0.446**	**0.662**	**0.497**	**0.444**		–
KNN	0.809	**0.88**	0.425	**0.654**	0.415	0.258		9.07
KM + KNN	0.833	0.88	0.421	0.653	0.411	0.225	2/0/4	9.64
CAC + KNN	**0.833**	0.88	**0.446**	0.653	**0.497**	**0.444**		–
SGD	0.831	0.85	0.272	0.531	0.248	0.438		15.8
KM + SGD	**0.857**	**0.88**	0.250	0.605	0.15	0.376	0/2/4	17.77
CAC + SGD	0.835	0.872	**0.375**	**0.62**	**0.497**	**0.472**		–
Ridge	0.828	0.847	0.223	0.495	0.112	0.329		31.76
KM + Ridge	0.858	**0.87**	0.372	0.581	0.124	0.356	0/2/5	18.13
CAC + Ridge	**0.858**	0.869	**0.446**	**0.62**	**0.497**	**0.444**		–
FNN	0.851	0.904	0.462	0.662	0.215	**0.496**		2.09
DMNN	0.853	0.906	**0.477**	**0.666***	0.262	0.485	1/2/3	0.44
CAC + DMNN	**0.854**	**0.907***	0.472	0.663	**0.281**	0.488		–
Total (9 w/o EC3)	1/3/5	2/5/1	0/1/8	1/1/7	0/0/9	0/2/7		
Avg. $\uparrow$F1 (%)	3.26/2.18	1.68/0.6	29.28/12.34	12.17/3.08	91.5/70.76	16.2/14.75		

7 Conclusion

Clustering has been effectively used with classification models in many previous models and for various applications. In this paper, we theoretically analyze potential reasons for the good performance of classifiers trained on underlying clusters in data. Our analysis yields insights into the benefits of using simple classifiers trained on clusters compared to simple or complex classifiers trained without clustering. Deeper analysis along these lines would be an interesting direction for future work.

Our analysis motivates the development of a new clustering framework, CAC, that enforces class separability within each cluster during cluster discovery. We design an efficient iterative algorithm to find such clusters. Unlike most previous approaches, CAC is simple, efficient, and can be used with any classifier. In our experiments, we find that CAC outperforms state-of-the-art methods of combined clustering and classification by 4.63%. This work can be extended in many ways, in part to address its current limitations, e.g., to perform multi-class classification or obtain non-linear class separation boundaries. More efficient methods to find the best classifiers on the clusters may also be investigated.

It would also be interesting to explore the benefits offered by unsupervised techniques in supervised applications like adversarial robustness, efficient regularization etc. Recent success of self supervised and contrastive learning approaches strongly suggests untapped potential that may be exploited by combining unsupervised with supervised learning.

References

[1] Qiang Qian, Songcan Chen, and Weiling Cai. Simultaneous clustering and classification over cluster structure representation. *Pattern Recognition*, 2012.

[2] Quanquan Gu and Jiawei Han. Clustered support vector machines. In *Artificial Intelligence and Statistics*, 2013.

[3] Tanmoy Chakraborty. Ec3: Combining clustering and classification for ensemble learning. In *ICDM*, 2017.

[4] Xiangrui Li, Dongxiao Zhu, and Phillip Levy. Predicting clinical outcomes with patient stratification via deep mixture neural networks. *AMIA Summits on Translational Science*, 2020.

[5] Ralph Snyderman. Personalized health care: from theory to practice. *Biotechnology journal*, 2012.

[6] Ahmed M Alaa, Jinsung Yoon, Scott Hu, and Mihaela Van der Schaar. Personalized risk scoring for critical care patients using mixtures of gaussian process experts. *arXiv preprint arXiv:1605.00959*, 2016.

[7] Harini Suresh, Jen J Gong, and John V Guttag. Learning tasks for multitask learning: Heterogenous patient populations in the icu. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018.

[8] Zina M Ibrahim, Honghan Wu, Ahmed Hamoud, Lukas Stappen, Richard JB Dobson, and Andrea Agarossi. On classifying sepsis heterogeneity in the icu: insight using machine learning. *Journal of the American Medical Informatics Association*, 2020.

[9] Tae-Kyun Kim and Roberto Cipolla. Mcboost: Multiple classifier boosting for perceptual co-clustering of images and visual features. *Advances in Neural Information Processing Systems*, 21:841–848, 2008.

[10] Mohammad Peikari, Sherine Salama, Sharon Nofech-Mozes, and Anne L Martel. A cluster-then-label semi-supervised learning approach for pathology image classification. *Scientific reports*, 2018.

[11] Izzat Alsmadi and Ikdam Alhami. Clustering and classification of email contents. *Journal of King Saud University-Computer and Information Sciences*, 2015.

[12] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of machine learning*. MIT Press, 2012.

[13] Manchek A Wong and JA Hartigan. Algorithm as 136: A k-means clustering algorithm. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, 1979.

[14] Sibylle Hess, Wouter Duivesteijn, and Decebal Mocanu. Softmax-based classification is k-means clustering: Formal proof, consequences for adversarial attacks, and improvement through centroid based tailoring. *arXiv:2001.01987*, 2020.

[15] Hind Elouedi, Walid Meliani, Zied Elouedi, and Nahla Ben Amor. A hybrid approach based on decision trees and clustering for breast cancer classification. In *SoCPaR*, 2014.

[16] Martin Vincent and Niels Richard Hansen. Sparse group lasso and high dimensional multinomial classification. *Computational Statistics & Data Analysis*, 2014.

[17] Andrea Visentin, Alessia Nardotto, and Barry O'Sullivan. Predicting judicial decisions: A statistically rigorous approach and a new ensemble classifier. In *ICTAI*, 2019.

[18] Ahmed Mahfouz, Abdullah Abuhussein, Deepak Venugopal, and Sajjan Shiva. Ensemble classifiers for network intrusion detection using a novel network attack dataset. *Future Internet*, 2020.

[19] William H Beluch, Tim Genewein, Andreas Nürnberger, and Jan M Köhler. The power of ensembles for active learning in image classification. In *CVPR*, 2018.

[20] Ivan Titov, Alexandre Klementiev, Kevin Small, and Dan Roth. Unsupervised aggregation for classification problems with large numbers of categories. In *AISTATS*, 2010.

[21] Botao Hao, Will Wei Sun, Yufeng Liu, and Guang Cheng. Simultaneous clustering and estimation of heterogeneous graphical models. *JMLR*, 2018.

[22] G. Qi, Q. Tian, and T. Huang. Locality-sensitive support vector machine by exploring local correlation and global regularization. In *CVPR*, 2011.

[23] Philip HS Torr. Locally linear support vector machines. In *ICML*, 2011.

[24] Zhouyu Fu, Antonio Robles-Kelly, and Jun Zhou. Mixing linear svms for nonlinear classification. *IEEE Transactions on Neural Networks*, 2010.

[25] Kilian Q Weinberger and Lawrence K Saul. Distance metric learning for large margin nearest neighbor classification. *JMLR*, 2009.

[26] Kun Song, Feiping Nie, Junwei Han, and Xuelong Li. Parameter free large margin nearest neighbor for distance metric learning. In *AAAI*, 2017.

[27] Hiroaki Sasaki, Takashi Takenouchi, Ricardo Monti, and Aapo Hyvarinen. Robust contrastive learning and nonlinear ica in the presence of outliers. In *UAI*, 2020.

[28] Conor Durkan, Iain Murray, and George Papamakarios. On contrastive learning for likelihood-free inference. In *ICML*, 2020.

[29] Weiling Cai, Songcan Chen, and Daoqiang Zhang. A simultaneous learning framework for clustering and classification. *Pattern Recognition*, 2009.

[30] Peter L Bartlett and Shahar Mendelson. Rademacher and gaussian complexities: Risk bounds and structural results. *JMLR*, 2002.

[31] Peter L Bartlett, Dylan J Foster, and Matus J Telgarsky. Spectrally-normalized margin bounds for neural networks. In *NeurIPS*, 2017.

[32] Kenji Kawaguchi, Leslie Pack Kaelbling, and Yoshua Bengio. Generalization in deep learning. *arXiv:1710.05468*, 2017.

[33] Pranjal Awasthi, Natalie Frank, and Mehryar Mohri. On the rademacher complexity of linear hypothesis sets. *arXiv preprint arXiv:2007.11045*, 2020.

[34] Stuart Lloyd. Least squares quantization in pcm. *IEEE transactions on information theory*, 1982.

[35] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, et al. Scikit-learn: Machine learning in Python. *JMLR*, 2011.

[36] Frank E. Harrell Jr. and Thomas Cason. Titanic. *OpenML*, 1994. URL https://www.openml.org/d/40945.

[37] M. Lichman. Uci repository. *UCI*, 2013. URL http://archive.ics.uci.edu/ml.

[38] I-Cheng Yeh and Che-hui Lien. The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, 2009.

[39] Ikaro Silva, George Moody, Daniel J Scott, Leo A Celi, and Roger G Mark. Predicting in-hospital mortality of icu patients: The physionet/computing in cardiology challenge 2012. *Computing in cardiology*, 2012.

[40] Alistair EW Johnson, Nic Dunkley, Louis Mayaud, Athanasios Tsanas, Andrew A Kramer, and Gari D Clifford. Patient specific predictions in the intensive care unit using a bayesian ensemble. In *CinC*, 2012.

[41] Jing Gao, Feng Liang, Wei Fan, Yizhou Sun, and Jiawei Han. A graph-based consensus maximization approach for combining multiple supervised and unsupervised models. *TKDE*, 2011.

[42] Xiang Ao, Ping Luo, Xudong Ma, Fuzhen Zhuang, Qing He, Zhongzhi Shi, and Zhiyong Shen. Combining supervised and unsupervised models via unconstrained probabilistic embedding. *Information Sciences*, 2014.

[43] Mayank. Ec3. *https://github.com/mayank953/EC3*, 2018.

[44] Sanjoy Dasgupta. *The hardness of k-means clustering*. University of California, 2008.

Appendix

A Proofs of Theorems

A.1 Proof of Theorem 1

Theorem 1. *Let $\mathcal{F}_j$ be a set of maps $x \mapsto f(x)$, and the function $q \mapsto \ell(q, y)$ be a λ-uniformly bounded function for any $q \in \{f(x) : f \in \mathcal{F}_j, x \in \mathcal{X}_j\}$ and $y \in \mathcal{Y}_j$. Then, for any $\delta > 0$, with probability at least $1 - \delta$ over an i.i.d. draw of m i.i.d. samples $((x_i, y_i))_{i=1}^m$, the following holds: for all maps $f \in \mathcal{F}_j$,*

$$\mathbb{E}_{x,y}[\ell(f(x), y)] \leq \frac{1}{m_j} \sum_{i=1}^{m_j} \ell(f(x_i), y_i) + 2\mathcal{R}_{m_j}(\ell \circ \mathcal{F}_j) + \lambda \sqrt{\frac{\ln(1/\delta)}{2m_j}}, \tag{2}$$

where $\mathcal{R}_{m_j}(\ell \circ \mathcal{F}_j) := \mathbb{E}_{S,\sigma}[\sup_{f \in \mathcal{F}_j} \frac{1}{m_j} \sum_{i=1}^{m_j} \sigma_i \ell(f(x_i), y_i)]$ where $\sigma_1, \ldots, \sigma_{m_j}$ are independent uniform random variables taking values in $\{-1, 1\}$.

Proof of Theorem 1. Let $S = ((x_i, y_i))_{i=1}^m$ and $S' = ((x'_i, y'_i))_{i=1}^m$. Define

$$\varphi(S) = \sup_{f \in \mathcal{F}_j} \mathbb{E}_{x,y}[\ell(f(x), y)] - \frac{1}{m} \sum_{i=1}^{m} \ell(f(x_i), y_i). \tag{3}$$

To apply McDiarmid's inequality to $\varphi(S)$, we compute an upper bound on $|\varphi(S) - \varphi(S')|$ where S and S' be two test datasets differing by exactly one point of an arbitrary index i_0; i.e., $S_i = S'_i$ for all $i \neq i_0$ and $S_{i_0} \neq S'_{i_0}$. Then,

$$\varphi(S') - \varphi(S) \leq \sup_{f \in \mathcal{F}_j} \frac{\ell(f(x_{i_0}), y_{i_0}) - \ell(f(x'_{i_0}), y'_{i_0})}{m} \leq \frac{\lambda}{m}. \tag{4}$$

Thus, by McDiarmid's inequality, for any $\delta > 0$, with probability at least $1 - \delta$,

$$\varphi(S) \leq \mathbb{E}_S[\varphi(S)] + \lambda \sqrt{\frac{\ln(1/\delta)}{2m}}. \tag{5}$$

Moreover,

$$\begin{aligned}
&\mathbb{E}_S[\varphi(S)] && (6)\\
&= \mathbb{E}_S \left[\sup_{f \in \mathcal{F}_j} \mathbb{E}_{S'} \left[\frac{1}{m} \sum_{i=1}^{m} \ell(f(x'_i), y'_i) \right] - \frac{1}{m} \sum_{i=1}^{m} \ell(f(x_i), y_i) \right] && (7)\\
&\leq \mathbb{E}_{S,S'} \left[\sup_{f \in \mathcal{F}_j} \frac{1}{m} \sum_{i=1}^{m} (\ell(f(x'_i), y'_i) - \ell(f(x_i), y_i) \right] && (8)\\
&\leq \mathbb{E}_{\xi,S,S'} \left[\sup_{f \in \mathcal{F}_j} \frac{1}{m} \sum_{i=1}^{m} \xi_i (\ell(f(x'_i), y'_i) - \ell(f(x_i), y_i)) \right] && (9)\\
&\leq 2\mathbb{E}_{\xi,S} \left[\sup_{f \in \mathcal{F}_j} \frac{1}{m} \sum_{i=1}^{m} \xi_i \ell(f(x_i), y_i)) \right] = 2\mathcal{R}_m(\ell \circ \mathcal{F}_j) && (10)
\end{aligned}$$

where the first line follows the definitions of each term, the second line uses the Jensen's inequality and the convexity of the supremum, and the third line follows that for each $\xi_i \in \{-1, +1\}$, the distribution of each term $\xi_i(\ell(f(x'_i), y'_i) - \ell(f(x_i), y_i))$ is the distribution of $(\ell(f(x'_i), y'_i) - \ell(f(x_i), y_i))$ since S and S' are drawn iid with the same distribution. The forth line uses the subadditivity of supremum. □

A.2 Proof of Theorem 3

Theorem 2. *The time complexity for each round of CAC is $O(ndk)$.*

Proof. For the time complexity of each round, it suffices to prove the running time for each point x_i is $O(dk)$. For each $j \in [k]$, we denote the centroid, positive centroid and negative centroid of $C_j \cup \{x_i\}$ as $\tilde{\mu}(C_j)$, $\tilde{\mu}^+(C_j)$ and $\tilde{\mu}^-(C_j)$ respectively. Given a cluster C_j and a point $z \in \mathbb{R}^d$, we denote the clustering cost of C w.r.t. z to be

$$g(C_j, z) := \sum_{x \in C_j} \|x - z\|^2.$$

A standard result from the k-means literature [44] is the following bias-variance decomposition of the k-means cost function:

$$g(C_j, z) = \phi(C_j, \mu(C_j)) + |C_j| \cdot \|\mu(C_j) - z\|^2.$$

Note that the above result holds for CAC cluster cost function also as can be shown easily by a little algebra. Based on this property, we have

$$\begin{aligned}
&\Gamma^+(C_j, x_i) \\
=&\|\tilde{\mu} - x_i\|^2 + |C_j| \cdot \|\tilde{\mu}(C_j) - \mu(C_j)\|^2 \\
&+ \alpha|C_j| \cdot \|\mu^+(C_j) - \mu^-(C_j)\|^2 \\
&- \alpha(|C_j| + 1) \cdot \|\tilde{\mu}^+(C_j) - \tilde{\mu}^-(C_j)\|^2 \\
&\Gamma^-(C_j, x_i) \\
=&- \|\mu(C_j) - x_i\|^2 - (|C_j| - 1)\|\tilde{\mu}(C_j) - \mu(C_j)\|^2 \\
&+ \alpha|C_j| \cdot \|\mu^+(C_j) - \mu^-(C_j)\|^2 \\
&- \alpha(|C_j| - 1) \cdot \|\tilde{\mu}^+(C_j) - \tilde{\mu}^-(C_j)\|^2
\end{aligned}$$

Then it suffices to prove that both $\Gamma^+(C_j, x_i)$ and $\Gamma^-(C_j, x_i)$ can be computed in $O(d)$ time, which implies $O(dk)$ time for computing $q := \operatorname{argmin}_j \Phi(x_i; C_p, C_j)$. Moreover, by the above formulations, it suffices to prove that $\tilde{\mu}(C_j)$, $\tilde{\mu}^+(C_j)$ and $\tilde{\mu}^-(C_j)$ can be computed in $O(d)$ time. Note that $\tilde{\mu}(C_p) = \frac{|C_p| \cdot \mu(C_p) - x_i}{|C_p| - 1}$ and $\tilde{\mu}(C_j) = \frac{|C_j| \cdot \mu(C_j) + x_i}{|C_j| + 1}$ for $j \neq p$. Hence, each $\tilde{\mu}(C_j)$ can be computed in $O(d)$ time. We discuss the following two cases for the computation of $\tilde{\mu}^+(C_j)$ and $\tilde{\mu}^-(C_j)$.

- Case $y_i = 1$. We have that $\tilde{\mu}^+(C_p) = \frac{\left(\sum_{x_l \in \mathcal{C}_p} y_l\right) \cdot \mu_p^+ - x_i}{\sum_{x_l \in \mathcal{C}_p} y_l - 1}$ and $\tilde{\mu}^-(C_p) = \mu^-(C_p)$. For $j \neq p$, we have that $\tilde{\mu}^+(C_j) \leftarrow \frac{\left(\sum_{x_l \in \mathcal{C}_j} y_l\right) \cdot \mu_j^+ + x_i}{\sum_{x_l \in \mathcal{C}_j} y_l + 1}$ and $\tilde{\mu}^-(C_j) = \mu^-(C_j)$.
- Case $y_i = 0$. We have that $\tilde{\mu}_p^- \leftarrow \frac{\left(\sum_{x_l \in \mathcal{C}_p} (1 - y_l)\right) \cdot \mu_p^- - x_i}{\sum_{x_l \in \mathcal{C}_p} (1 - y_l) - 1}$ and $\tilde{\mu}^+(C_p) = \mu^+(C_p)$. For $j \neq p$, we have that $\tilde{\mu}_j^- \leftarrow \frac{\left(\sum_{x_l \in \mathcal{C}_j} (1 - y_l)\right) \cdot \mu_j^- + x_i}{\sum_{x_l \in \mathcal{C}_j} (1 - y_l) + 1}$ and $\tilde{\mu}^+(C_j) = \mu^+(C_j)$.

By the above update rules, both $\tilde{\mu}^+(C_j)$ and $\tilde{\mu}^-(C_j)$ can be computed in $O(d)$ time, which completes the proof. □

A.3 Proof for Lemma 1

Lemma 1. *For d dimentional vectors $\mathbf{u}, \mathbf{v}$ and $\boldsymbol{\beta}$, $\frac{\partial \Delta}{\partial \|\mathbf{u}\|} < 0$ for $\Delta = A - (B\boldsymbol{\beta}\mathbf{u} + C\boldsymbol{\beta}\mathbf{v})$ if $\beta_i u_i > 0, \forall i \in \{1, 2, \cdots, d\}$ and $A, B > 0$.*

Proof. Let u_i be the ith element of $\mathbf{u}$. Expanding $\frac{\partial \Delta}{\partial \|\mathbf{u}\|}$ using chain rule, we get

$$\frac{\partial \Delta}{\partial \|\mathbf{u}\|} = \frac{\partial \Delta}{\partial \mathbf{u}} \cdot \frac{\partial \mathbf{u}}{\partial \|\mathbf{u}\|} = -B\boldsymbol{\beta} \cdot \frac{\partial \mathbf{u}}{\partial \|\mathbf{u}\|} \text{ (Since } \frac{\partial \Delta}{\partial \mathbf{u}} = -B\boldsymbol{\beta}\mathbf{u})$$

Let u_i denote the ith element of $\mathbf{u}$. Then, from the definition of vector derivatives we have,

$$\begin{aligned}\frac{\partial \mathbf{u}}{\partial \|\mathbf{u}\|} &= \left[\frac{\partial u_1}{\partial \|\mathbf{u}\|}, \frac{\partial u_2}{\partial \|\mathbf{u}\|}, \cdots, \frac{\partial u_d}{\partial \|\mathbf{u}\|}\right] \\ &= \left[\left(\frac{\partial \|\mathbf{u}\|}{\partial u_1}\right)^{-1}, \left(\frac{\partial \|\mathbf{u}\|}{\partial u_2}\right)^{-1}, \cdots, \left(\frac{\partial \|\mathbf{u}\|}{\partial u_d}\right)^{-1}\right] \\ &= \|\mathbf{u}\| \left[\frac{1}{u_1}, \frac{1}{u_2}, \cdots, \frac{1}{u_d}\right]\end{aligned}$$

Thus, $\frac{\partial \Delta}{\partial \|\mathbf{u}\|} = -B \cdot \|\mathbf{u}\| \cdot \sum_{i=1}^{d} \frac{\beta_i}{u_i}$, where $u_i = \mu_i^+ - \mu_i^-$. Since $\beta \cdot \mu^+ > 0$ and $\beta \cdot \mu^- < 0$, it implies that $\beta \cdot \mathbf{u} > 0$. Assuming that every dimension of $\mathbf{u}$ lies on the same side of regression plane β as the original point, i.e. $\beta_i u_i = \beta_i > 0, \forall i$, $\frac{\beta_i}{u_i} > 0 \; \forall i$, then $\frac{\partial \Delta}{\partial \|\mathbf{u}\|} < 0$ as B and $\|\mathbf{u}\|$. Thus under above assumption, we prove that Δ decreases as $\|\mathbf{u}\|$ increases.

□

A.4 Proof for Theorem 4

Theorem 3. *Let $X = (x_1, x_2, \cdots, x_N)$ be a training dataset with n records $x_i \in \mathbb{R}^d$ together with binary labels $y_i \in \{0, 1\}$. Define the log-loss over the entire dataset as :*

$$\ell(X) = -\Sigma_i y_i \ln(p(x_i)) + (1 - y_i) \ln(1 - p(x_i))$$

where $p(x_i) = [1 + \exp(-\beta x_i)]^{-1}$ and $\beta = \operatorname{argmin}_\beta \ell(X)$. Then,

$$C_1 - (C_3 \beta \mu^+ - C_4 \beta \mu^-) \leq \ell(X) \leq C_2 - (C_3 \beta \mu^+ - C_4 \beta \mu^-)$$

where $C_1 = N \ln(2)$, $C_2 = N \ln(1 + \exp(c)) - \frac{Nc}{2}$, $C_3 = \frac{N^+}{2}$, $C_4 = \frac{N^-}{2}$, $N^+ = \sum_{x_i \in X} y_i$, $N^- = \sum_{x_i \in X} (1 - y_i)$ and $c = \operatorname{argmax}_i \|\beta x_i\|$.

Proof. **Lower Bound** Let $\ell(X, y) = \Sigma_{y_i=1} \ell^+(x_i) + \Sigma_{y_i=0} \ell^-(x_i)$ s.t.

$$\ell^+(x_i) = -\ln(p_i), \; \ell^-(x_i) = -\ln(1 - p_i) \tag{11}$$

Applying Jensen's inequality on ℓ^+ and ℓ^-,

$$\begin{aligned}\ell(X) =& \Sigma_{x_i \in X^+} \ell^+(x_i) + \Sigma_{x_i \in X^-} \ell^-(x_i) \\ =& -\Sigma_{y_i=1} \ln[1 + \exp(-\beta x_i)]^{-1} - \Sigma_{y_i=0} \ln[1 - (1 + \exp(-\beta x_i))^{-1}] \quad \text{(By 11)} \\ \geq& -N^+ \ln\left[1 + \exp\left(-\beta \cdot \left(\frac{\Sigma_{x_i \in X^+} x_i}{N^+}\right)\right)\right]^{-1} \\ & - N^- \ln\left[1 - \left(1 + \exp\left(-\beta \cdot \left(\frac{\Sigma_{x_i \in X^-} x_i}{N^-}\right)\right)\right)^{-1}\right] \quad \text{(Jensen's inequality for convex functions)} \\ \geq& N^+ \ln[1 + \exp(-\beta \mu^+)] - N^- \ln\left[\frac{\exp(-\beta \mu^-)}{1 + \exp(-\beta \mu^-)}\right] \quad \text{(Substituting } \mu^+ \text{ and } \mu^-\text{)} \\ \geq& N^+ \ln[1 + \exp(-\beta \mu^+)] + N^- \ln[1 + \exp(\beta \mu^-)]\end{aligned}$$

We observe that $\ln(1 + \exp(x)) \geq \ln(2) + \frac{x}{2} \; \forall x \in \mathbb{R}$ by considering the Taylor series expansion of $\ln(1 + \exp(x))$ at $x = 0$. $\ln(1 + \exp(x)) = \ln(2) + \frac{x}{2} + \frac{x^2}{8} + \mathcal{O}(x^4)$. Then,

$$\begin{aligned}\ell(X) &\geq N^+ \left(\ln(2) - \frac{\beta \mu^+}{2}\right) + N^- \left(\ln(2) + \frac{\beta \mu^-}{2}\right) \\ &\geq N \ln(2) - \frac{1}{2} \cdot (N^+ \beta \mu^+ - N^- \beta \mu^-)\end{aligned}$$

where $C_1 = N\ln(2), C_3 = \frac{N^+}{2}, C_4 = \frac{N^-}{2}$.

Upper Bound

Since all points x_i are constrained within their respective clusters, we note that the maximum log-loss for a point x_i labelled as y_i is not unbounded. Let $c = \text{argmax}_i \|\beta x_i\|$. Since ℓ^+ is a monotonically decreasing function and ℓ^- is monotonically increasing, we can bound them using linear functions s^+ and s^- respectively. $s^+ \geq \ell^+$ and $s^- \geq \ell^-$ for all points X^+ and X^- respectively. $s^+(x_i)$ passes through $(-c, \ln(p(-c)))$ and $(c, \ln(p(c)))$. Hence $s^+(x) = K_1 - K_2 \cdot \beta x$ where $K_1, K_2 > 0$. Similarly, $s^-(x) = K_3 + K_4 \cdot \beta x$ where $K_3, K_4 > 0$.

Solving for $K_1 \cdots K_4$, we get

$$K_1 = K_3 = \ln(1 + \exp(c)) - \frac{c}{2},\ K_2 = K_4 = \frac{1}{2}$$

Then,

$$\begin{aligned}
\ell(X) &= \Sigma_{x_i \in X^+} \ell^+(x_i) + \Sigma_{x_i \in X^-} \ell^-(x_i) \\
&< \Sigma_{x_i \in X^+} s^+(x_i) + \Sigma_{x_i \in X^-} s^-(x_i) \\
&\leq N^+ s^+ \left(\frac{\Sigma_{x_i \in X^+} x_i}{N^+} \right) + N^- s^- \left(\frac{\Sigma_{x_i \in X^-} x_i}{N^-} \right) && \text{(Jensen's inequality for a linear function)} \\
&\leq N^+ s^+(\mu^+) + N^- s^-(\mu^-) \\
&\leq N^+ (K_1 - K_2 \cdot \beta\mu^+) + N^- (K_3 + K_4 \cdot \beta\mu^-) \\
&\leq C_2 - (C_3 \beta\mu^+ - C_4 \beta\mu^-) && \text{(Rearranging terms)}
\end{aligned}$$

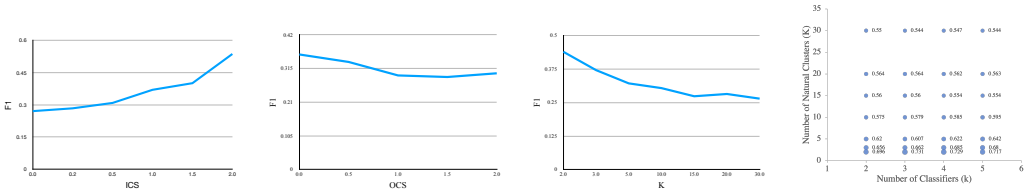
where $C_2 = N\ln(1 + \exp(c)) - \frac{Nc}{2}$ and $C_3 = \frac{N^+}{2}, C_4 = \frac{N^-}{2}$. □

B Supplementary Experimental Results and Discussions

B.1 Results on Synthetic Dataset

We choose Logistic Regression as the base classifier for CAC while testing its performance on the synthetic dataset as described in section 6.1. The range of these parameters in our simulations are as follows: $ICS \in \{0, 0.2, 0.5, 1, 1.5, 2\}$, $OCS \in \{1, 1.5, 2\}$, $K \in \{2, 3, 5, 10, 15, 20, 30\}$ and $k \in \{2, 3, 4, 5\}$. We run CAC+LR for all combination of parameters.

Figure 3 presents the results of the experiments. In each subfigure, the results are averaged over all the parameters not shown in the axes. In Figure 3a, note that the performance of CAC improves as Inner Class Separation within the clusters increases. This is in line with our hypothesis that if the classes are well separated within the clusters, then it will improve the performance of the classifiers. The performance of CAC does not depend much on the Outer Cluster Separation (Figure 3b). This is also expected as a classifier will be relatively unaffected by the presence of another cluster, irrespective of its distance as the data in that cluster is not a part of its training set.



(a) Effect of Inter Class Separation (ICS) on F1 and AUC.

(b) Effect of Outer Cluster Separation (OCS) on F1 and AUC.

(c) Effect of number of natural clusters (K) on F1 and AUC.

(d) AUC with varying #classifiers (k) and Natural #Clusters (K).

Figure 3: Evaluating the effect of different parameters on the performance of CAC

As the number of natural clusters in the dataset increases, the performance of CAC decreases (Figure 3c). This phenomenon is expected from linear classifiers as they cannot handle nonlinearities in

data. Hence, dividing the data into multiple groups will decrease the piece-wise nonlinearity and the overall algorithm will work better [2].

In Figure 3d, we observe that the results are better for the cases when $k \geq K$ (i.e., closer to the points near the x-axis). This shows that the performance is better when the number of clusters given as input is not less than the natural clusters in the data, compared to the case when the input number of clusters is less than the natural number of clusters. In the latter case, each (linear) classifier has to learn a non-linear boundary due to the intrinsic cluster structure that leads to a decrease in the performance.

B.2 Hyperparameter Details

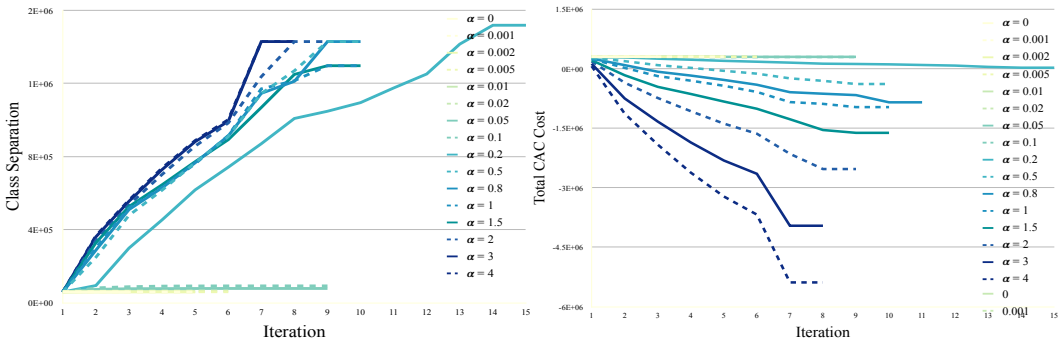
Table 3 shows the tuned values of α used by different variants of CAC on different datasets. We find these values by performing a 5-fold Grid search cross validation.

Table 3: Tuned α values used for CAC in experiments found after 5-fold GridCV search

Classifier	Titanic	Magic	Credit	Adult	Diabetes	CIC
LR	0.8	0.01	0.02	0.05	2.5	0.05
SVM	0.01	0.02	0.15	0.15	2.5	0.5
LDA	0.08	0.05	0.02	0.15	2.5	0.05
Perceptron	0.5	0.01	0.2	0.15	2.5	0.5
RF	0.05	0.02	0.15	0.02	2.5	0.5
KNN	0.01	0.01	0.15	0.02	2.5	0.5
SGD	0.3	0.01	0.08	0.15	2.5	0.01
Ridge	0.01	0.02	0.15	0.15	2.5	0.5
DMNN	0.2	0.01	0.3	0.8	0.15	0.1

B.3 Analyzing the CAC cost function

Figure 4a shows how the class separation part of the CAC cost ($\phi = \mathcal{D} - \alpha \cdot \mathcal{S}$) increases with every iteration. Note that as α increases, $\mathcal{S}$ also increases at an increasing rate. Also note how $\mathcal{S}$ is monotonically increasing thereby supporting our conjecture mentioned in section 2.



(a) Total Class separation v/s iterations of CAC for different α.

(b) Total CAC cost v/s iterations for different α.

Figure 4: CAC Training stats with Base classifier: LR and Dataset: Adult

As proved in Theorem 2, the CAC cost function is monotonically decreasing (See Figure 4b).

B.4 Extended Results on Real Datasets

We present a detailed evaluation of the performance of CAC by conducting the same se of experiments in a 5-fold cross validation manner in Table 4. In Table 5, we present the 5-fold averaged results of experiments conducted on a held-out test set. We report the standard deviations in F1 scores also

for both set of experiments. Note that the performance of CAC follows similar trends in all the experiments. There is consistent improvement over the base classifier and k-means + base classifier approach on all the datasets and across different classifiers.

Table 4: 5 Fold-CV F1 score of each method on different datasets. $*/*/*$ indicates the number of times the base classifier, KM+Base and CAC+Base attains the best performance.

Classifier	Titanic	Magic	Credit	Adult	Diabetes	CIC	Total	Avg. $\uparrow$ F1(%)
EC3	0.812	0.850	$\mathbf{0.503}^*$	$\mathbf{0.652}^*$	NA	0.495	2	
LR	$\mathbf{0.804 \pm 0.034}$	0.848 ± 0.004	0.362 ± 0.007	0.557 ± 0.012	0.116 ± 0.006	0.454 ± 0.034		20.57
KM + LR	0.65 ± 0.037	$\mathbf{0.857 \pm 0.003}$	0.496 ± 0.005	0.616 ± 0.009	0.5 ± 0.008	$\mathbf{0.491 \pm 0.008}$	1/2/3	4.9
CAC + LR	0.801 ± 0.021	0.857 ± 0.003	$\mathbf{0.498 \pm 0.005}$	$\mathbf{0.636 \pm 0.009}$	$\mathbf{0.504 \pm 0.005}^*$	0.491 ± 0.005		-
SVM	0.803 ± 0.032	0.847 ± 0.005	0.273 ± 0.008	0.532 ± 0.01	0.108 ± 0.005	0.413 ± 0.031		20.83
KM + SVM	$\mathbf{0.825 \pm 0.035}$	$\mathbf{0.88 \pm 0.005}$	$\mathbf{0.406 \pm 0.006}$	0.596 ± 0.011	0.108 ± 0.018	0.421 ± 0.023	0/3/3	11.12
CAC + SVM	0.825 ± 0.035	0.88 ± 0.005	0.399 ± 0.004	$\mathbf{0.616 \pm 0.013}$	$\mathbf{0.425 \pm 0.157}$	$\mathbf{0.451 \pm 0.009}$		-
LDA	0.802 ± 0.031	0.845 ± 0.003	0.374 ± 0.007	0.528 ± 0.01	0.118 ± 0.007	$\mathbf{0.491 \pm 0.023}$		15.26
KM + LDA	$\mathbf{0.825 \pm 0.016}$	0.87 ± 0.005	$\mathbf{0.428 \pm 0.007}$	0.588 ± 0.014	0.324 ± 0.195	0.489 ± 0.008	1/2/3	3.29
CAC + LDA	0.825 ± 0.016	$\mathbf{0.871 \pm 0.005}$	0.426 ± 0.008	$\mathbf{0.613 \pm 0.017}$	$\mathbf{0.42 \pm 0.166}$	0.485 ± 0.016		-
Perceptron	0.682 ± 0.077	0.735 ± 0.046	0.382 ± 0.058	0.503 ± 0.059	0.447 ± 0.037	0.394 ± 0.025		13.08
KM + Perceptron	0.755 ± 0.037	0.793 ± 0.038	0.384 ± 0.044	0.484 ± 0.123	0.422 ± 0.059	0.407 ± 0.04	0/0/6	9.52
CAC + Perceptron	$\mathbf{0.795 \pm 0.022}$	$\mathbf{0.844 \pm 0.021}$	$\mathbf{0.43 \pm 0.019}$	$\mathbf{0.55 \pm 0.145}$	$\mathbf{0.484 \pm 0.039}$	$\mathbf{0.451 \pm 0.009}$		-
RF	$\mathbf{0.805 \pm 0.037}$	0.893 ± 0.008	$\mathbf{0.408 \pm 0.011}$	0.648 ± 0.006	0.406 ± 0.006	0.326 ± 0.022		5.36
KM + RF	0.796 ± 0.035	$\mathbf{0.896 \pm 0.005}$	0.404 ± 0.015	0.648 ± 0.007	0.405 ± 0.003	0.296 ± 0.02	2/1/3	6.62
CAC + RF	0.797 ± 0.037	0.895 ± 0.006	0.398 ± 0.011	$\mathbf{0.649 \pm 0.005}$	$\mathbf{0.484 \pm 0.04}$	$\mathbf{0.45 \pm 0.009}$		-
KNN	$\mathbf{0.79 \pm 0.021}$	$\mathbf{0.883 \pm 0.003}$	$\mathbf{0.42 \pm 0.005}$	0.645 ± 0.004	0.413 ± 0.015	0.256 ± 0.015		7.66
KM + KNN	0.788 ± 0.021	0.883 ± 0.002	0.42 ± 0.006	$\mathbf{0.646 \pm 0.004}$	0.405 ± 0.011	0.243 ± 0.021	3/1/2	8.36
CAC + KNN	0.788 ± 0.021	0.883 ± 0.002	0.42 ± 0.007	0.645 ± 0.004	$\mathbf{0.482 \pm 0.043}$	$\mathbf{0.45 \pm 0.009}$		-
SGD	$\mathbf{0.797 \pm 0.027}$	0.845 ± 0.007	0.363 ± 0.08	0.545 ± 0.017	0.22 ± 0.08	$\mathbf{0.437 \pm 0.046}$		11.19
KM + SGD	0.775 ± 0.065	0.873 ± 0.006	0.387 ± 0.078	0.596 ± 0.028	0.204 ± 0.046	0.409 ± 0.036	2/0/4	9.93
CAC + SGD	0.796 ± 0.022	$\mathbf{0.878 \pm 0.006}$	$\mathbf{0.414 \pm 0.056}$	$\mathbf{0.607 \pm 0.026}$	$\mathbf{0.437 \pm 0.132}$	0.434 ± 0.026		-
Ridge	0.801 ± 0.028	0.845 ± 0.004	0.245 ± 0.005	0.483 ± 0.015	0.108 ± 0.005	0.35 ± 0.024		25.53
KM + Ridge	$\mathbf{0.823 \pm 0.036}$	$\mathbf{0.871 \pm 0.004}$	$\mathbf{0.383 \pm 0.011}$	0.581 ± 0.013	0.108 ± 0.018	0.368 ± 0.021	0/3/3	13.43
CAC + Ridge	0.823 ± 0.036	0.871 ± 0.005	0.376 ± 0.012	$\mathbf{0.61 \pm 0.018}$	$\mathbf{0.425 \pm 0.157}$	$\mathbf{0.45 \pm 0.009}$		-
FNN	0.848 ± 0.007	0.904 ± 0.001	0.469 ± 0.026	$\mathbf{0.666 \pm 0.007}$	0.216 ± 0.025	$\mathbf{0.507 \pm 0.012}$		-1.51
DMNN	$\mathbf{0.857 \pm 0.003}$	$\mathbf{0.907 \pm 0.001}$	0.47 ± 0.012	0.662 ± 0.009	0.217 ± 0.025	0.478 ± 0.015	2/2/2	4.91
CAC + DMNN	0.857 ± 0.005	0.905 ± 0.0	$\mathbf{0.471 \pm 0.006}$	0.664 ± 0.005	$\mathbf{0.251 \pm 0.021}$	0.476 ± 0.011		
Total (9 w/o EC3)	4/4/1	2/3/4	0/2/7	1/1/7	0/0/9	2/1/6		
Avg. $\uparrow$F1 (%)	2.54/3.33	3.44/0.81	18.98/1.45	10.99/3.7	89.26/47.74	17.44/17.44		

Table 5: F1 of each method on different datasets evaluated 5 times on a separate held out testing dataset. $*/*/*$ indicates the number of times the base classifier, KM+Base and CAC+Base attains the best performance. Highlighted values signify the best result in that category while the overall best classifier for a dataset is denoted by *. The Avg. $\uparrow$ F1 denotes the average % increase in F1 score of CAC over Base and k-means variants across different datasets (column-wise).

Classifier	Titanic	Magic	Credit	Adult	Diabetes	CIC	Total	Avg. $\uparrow$ F1(%)
EC3	0.845 ± 0.003	0.857 ± 0.001	$\mathbf{0.514 \pm 0.001}^*$	0.661 ± 0.001	NA	$\mathbf{0.498 \pm 0.006}^*$	2	
LR	$\mathbf{0.839 \pm 0.0}$	0.852 ± 0.0	0.336 ± 0.0	0.556 ± 0.0	0.12 ± 0.0	0.45 ± 0.0		20.77
KM + LR	0.674 ± 0.0	$\mathbf{0.861 \pm 0.0}$	$\mathbf{0.506 \pm 0.0}$	0.619 ± 0.0	$\mathbf{0.503 \pm 0.0}^*$	$\mathbf{0.467 \pm 0.001}$	1/4/1	4.9
CAC + LR	0.838 ± 0.0	0.859 ± 0.0	0.506 ± 0.0	$\mathbf{0.642 \pm 0.0}$	0.497 ± 0.0	0.466 ± 0.002		
SVM	0.84 ± 0.0	0.852 ± 0.0	0.249 ± 0.0	0.532 ± 0.0	0.112 ± 0.0	0.406 ± 0.002		25.31
KM + SVM	$\mathbf{0.862 \pm 0.0}^*$	$\mathbf{0.879 \pm 0.0}$	0.399 ± 0.0	0.601 ± 0.0	0.125 ± 0.0	0.441 ± 0.001	0/2/4	13.34
CAC + SVM	0.862 ± 0.0	0.878 ± 0.0	$\mathbf{0.447 \pm 0.0}$	$\mathbf{0.62 \pm 0.0}$	$\mathbf{0.497 \pm 0.0}$	$\mathbf{0.444 \pm 0.0}$		-
LDA	0.828 ± 0.0	0.846 ± 0.0	0.358 ± 0.0	0.531 ± 0.0	0.12 ± 0.0	0.446 ± 0.0		19.69
KM + LDA	$\mathbf{0.857 \pm 0.0}$	$\mathbf{0.87 \pm 0.0}$	$\mathbf{0.423 \pm 0.0}$	0.592 ± 0.0	0.45 ± 0.0	$\mathbf{0.486 \pm 0.002}$	0/4/2	1.82
CAC + LDA	0.857 ± 0.0	0.87 ± 0.0	0.421 ± 0.0	$\mathbf{0.62 \pm 0.0}$	$\mathbf{0.497 \pm 0.0}$	0.48 ± 0.0		-
Perceptron	0.729 ± 0.0	0.83 ± 0.0	0.367 ± 0.0	0.482 ± 0.0	0.469 ± 0.0	0.378 ± 0.0		12.38
KM + Perceptron	$\mathbf{0.837 \pm 0.0}$	0.826 ± 0.03	0.397 ± 0.057	0.568 ± 0.014	0.407 ± 0.0	0.357 ± 0.069	0/1/5	7.84
CAC + Perceptron	0.831 ± 0.0	$\mathbf{0.85 \pm 0.0}$	$\mathbf{0.414 \pm 0.0}$	$\mathbf{0.62 \pm 0.0}$	$\mathbf{0.497 \pm 0.0}$	$\mathbf{0.446 \pm 0.0}$		-
RF	0.834 ± 0.009	$\mathbf{0.891 \pm 0.004}$	0.409 ± 0.011	0.655 ± 0.007	0.411 ± 0.002	0.313 ± 0.019		7.03
KM + RF	$\mathbf{0.852 \pm 0.019}$	0.89 ± 0.002	0.39 ± 0.007	0.654 ± 0.005	0.408 ± 0.006	0.296 ± 0.023	1/1/4	7.74
CAC + RF	0.826 ± 0.015	0.886 ± 0.004	$\mathbf{0.446 \pm 0.0}$	$\mathbf{0.661 \pm 0.005}$	$\mathbf{0.497 \pm 0.0}$	$\mathbf{0.444 \pm 0.0}$		-
KNN	0.809 ± 0.0	$\mathbf{0.88 \pm 0.0}$	0.425 ± 0.0	$\mathbf{0.654 \pm 0.0}$	0.415 ± 0.0	0.258 ± 0.0		9.07
KM + KNN	$\mathbf{0.833 \pm 0.0}$	0.88 ± 0.0	0.421 ± 0.0	0.653 ± 0.0	0.411 ± 0.001	0.225 ± 0.0	2/1/3	9.64
CAC + KNN	0.833 ± 0.0	0.88 ± 0.0	$\mathbf{0.446 \pm 0.0}$	0.653 ± 0.0	$\mathbf{0.497 \pm 0.0}$	$\mathbf{0.444 \pm 0.0}$		-
SGD	0.793 ± 0.043	0.848 ± 0.003	0.291 ± 0.116	0.552 ± 0.011	0.201 ± 0.072	$\mathbf{0.449 \pm 0.019}$		15.16
KM + SGD	0.78 ± 0.051	0.878 ± 0.003	$\mathbf{0.414 \pm 0.038}$	0.599 ± 0.023	0.209 ± 0.053	0.443 ± 0.019	1/2/4	8.61
CAC + SGD	$\mathbf{0.831 \pm 0.005}$	$\mathbf{0.879 \pm 0.002}$	0.356 ± 0.109	$\mathbf{0.62 \pm 0.0}$	$\mathbf{0.497 \pm 0.0}$	0.426 ± 0.026		-
Ridge	0.828 ± 0.0	0.847 ± 0.0	0.223 ± 0.0	0.495 ± 0.0	0.112 ± 0.0	0.329 ± 0.0		31.76
KM + Ridge	$\mathbf{0.858 \pm 0.0}$	$\mathbf{0.87 \pm 0.0}$	0.371 ± 0.0	0.581 ± 0.0	0.124 ± 0.0	0.357 ± 0.001	0/2/4	18.13
CAC + Ridge	0.858 ± 0.0	0.869 ± 0.0	$\mathbf{0.446 \pm 0.0}$	$\mathbf{0.62 \pm 0.0}$	$\mathbf{0.497 \pm 0.0}$	$\mathbf{0.444 \pm 0.0}$		-
FNN	0.851 ± 0.007	0.904 ± 0.001	0.459 ± 0.006	$\mathbf{0.666 \pm 0.007}^*$	0.234 ± 0.027	0.477 ± 0.016		0.53
DMNN	0.849 ± 0.005	0.905 ± 0.001	0.466 ± 0.007	0.662 ± 0.009	0.253 ± 0.019	$\mathbf{0.485 \pm 0.011}$	1/1/4	-0.28
CAC + DMNN	$\mathbf{0.857 \pm 0.003}$	$\mathbf{0.907 \pm 0.0}^*$	$\mathbf{0.47 \pm 0.005}$	0.664 ± 0.005	$\mathbf{0.255 \pm 0.027}$	0.457 ± 0.017		-
Total (9 w/o EC3)	1/6/2	2/4/3	0/3/6	2/0/7	0/1/8	1/3/5		
Avg. $\uparrow$F1 (%)	3.3/2.68	1.63/0.23	26.88/4.28	11.78/3.58	92.62/46.42	15.38/13.92		

C Explaining CAC on Real Datasets

In Figure 5, we note that the total ICS value in the clusters found by the k-means algorithm is minimum for the Magic dataset. after running CAC on Magic dataset, the ICS value does not improve significantly. The Diabetes dataset also has a low ICS(KM) value but ICS after running CAC is very high. This gives a hint to why CAC performs the best on Diabetes dataset while it does not lead to significant improvements on the Magic dataset.

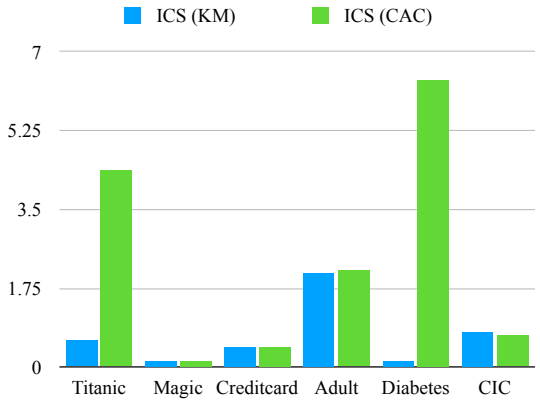


Figure 5: The total Inter Cluster Separation (ICS) for clusters found by k-means and CAC algorithms. α for CAC is set to the optimum values used by the CAC + LR variant.